

Expt 7	Spectral Estimation
Date:	7.1 Power Spectral Density Estimation and Spectral Analysis of Noisy Sinusoidal Signals Using Periodogram Method in MATLAB

Code:

```

clc;

clear;

close all;

%% Signal

fs = 1000;

t = (0:999)/fs;

s = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t); % Original signal

n = 0.3*randn(size(t)); % Noise

x = s + n; % Noisy signal

%% Plot Signal with Noise

figure;

plot(t, x, 'b');

grid on;

title('Signal with Noise');

xlabel('Time (s)');

ylabel('Amplitude');

%% PSD using Periodogram

figure;

periodogram(x, [], [], fs);

title('PSD using Periodogram');

%% Magnitude Spectrum using FFT

```

```

X = abs(fft(x));

f = (0:length(X)-1) * fs / length(X);

figure;

plot(f(1:500), X(1:500), 'LineWidth', 1.2);

grid on;

title('Magnitude Spectrum');

xlabel('Frequency (Hz)');

ylabel('|X(f)|');

```

```

%% Power Comparison (Signal vs Noise)

figure;

bar([mean(s.^2) var(n)]);

set(gca, 'XTickLabel', {'Signal','Noise'});

grid on;

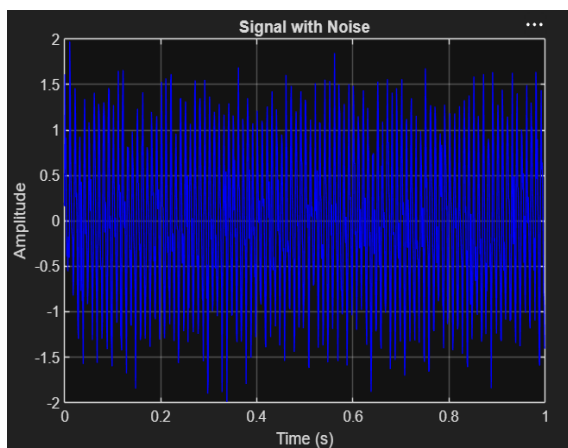
title('Power Comparison');

ylabel('Power');

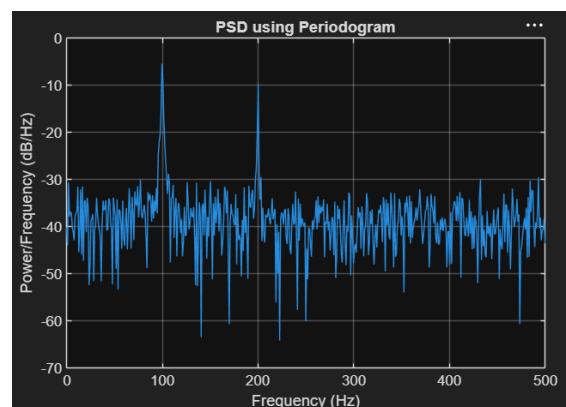
```

Output:

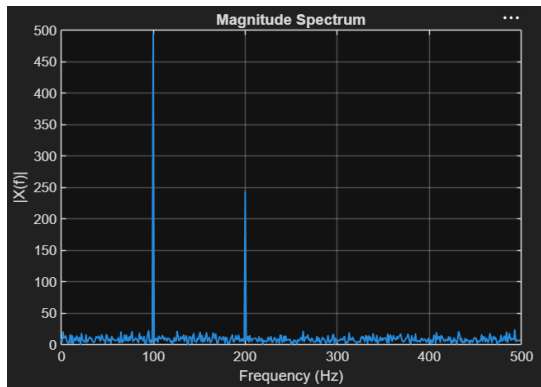
Observation 1:



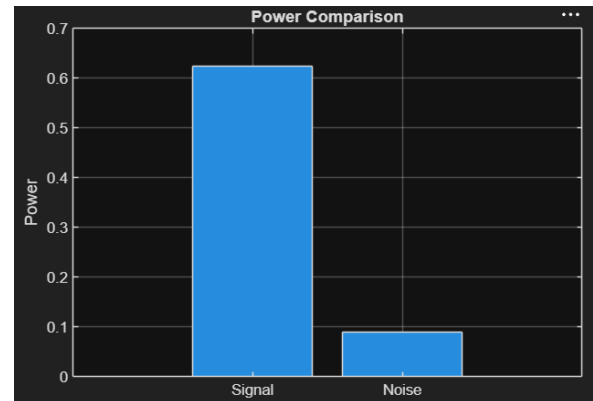
Observation 2:



Observation 3:



Observation 4:



Expt 7	Spectral Estimation
Date:	7.2 Analysis of Windowing Effects on Spectral Estimation using Rectangular , Hamming and Hanning Windows in MATLAB

Code:

```
clc; clear; close all;
```

```
% Signal
```

```
fs = 1000; N = 256; t = (0:N-1)/fs;
```

```
x = sin(2*pi*100*t) + 0.8*sin(2*pi*110*t);
```

```
NFFT = 2048; f = (0:NFFT/2-1)*(fs/NFFT);
```

```
% Windows
```

```
w = {rectwin(N), hamming(N), hann(N)};
```

```
names = {'Rectangular','Hamming','Hanning'};
```

```
% Processing
```

```
for i = 1:3
```

```
    xw = x(:).*w{i};
```

```
    X = fft(xw,NFFT);
```

```
P{i} = abs(X(1:NFFT/2));
```

```
P{i} = P{i}/max(P{i});
```

```
end
```

```
%% Individual Spectrums
```

```
figure;
```

```
for i = 1:3
```

```
    subplot(3,1,i);
```

```
    plot(f,20*log10(P{i}),'LineWidth',1.5);
```

```
    title(['Spectrum using ' names{i} ' Window']);
```

```
    xlim([0 250]); grid on;
```

```
end
```

```
%% Comparison
```

```
figure;
```

```
plot(f,20*log10(P{1}),'k',f,20*log10(P{2}),'r',f,20*log10(P{3}),'b','LineWidth',1.5);
```

```
legend(names); title('Window Comparison'); xlim([50 170]); grid on;
```

```
%% Window Shapes
```

```
figure;
```

```
for i = 1:3
```

```
    subplot(3,1,i); plot(w{i},'LineWidth',1.5);
```

```
    title([names{i} ' Window']); grid on;
```

```
end
```

```
%% Peak Detection
```

```
for i = 1:3
```

```
    [pks{i},loc{i}] = findpeaks(P{i},f,'MinPeakHeight',0.3);
```

```
    fprintf('\n%s Window Frequencies:\n',names{i});
```

```

disp(loc{i});

end

%% Peak Marking (Hamming)

figure;

plot(f,20*log10(P{2}),'LineWidth',1.5); hold on;

plot(loc{2},20*log10(pks{2}),'ro','MarkerFaceColor','r');

title('Detected Frequencies (Hamming)'); xlim([50 170]); grid on;

%% Main Peaks

fprintf('\nMain Spectral Peaks:\n');

for i = 1:3

    [~,idx] = max(P{i});

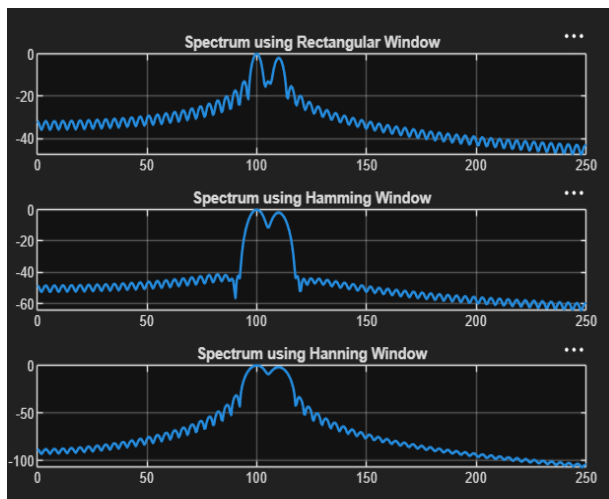
    fprintf('%s Window Peak = %.2f Hz\n',names{i},f(idx));

end

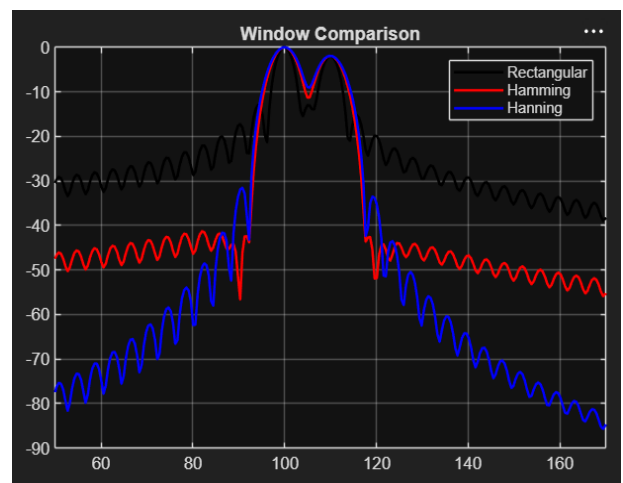
```

Output:

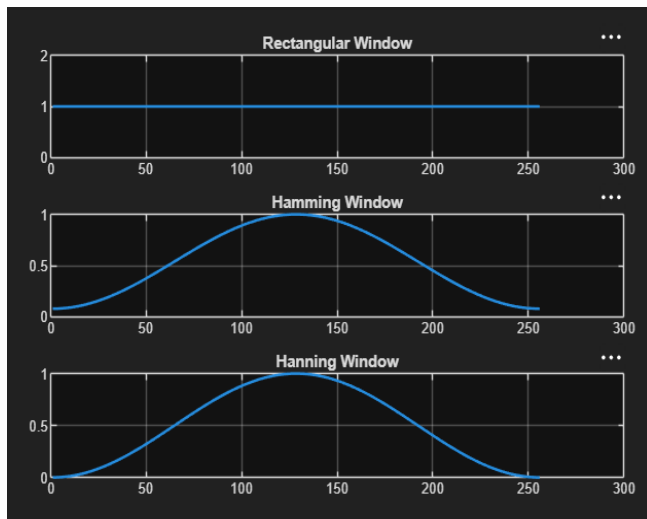
Objective 1:



Objective 2:



Objective 3:



Objective 4:

